

JAVA WITH DSA-DAY4

ARRAYS

MOVE ZEROES TO END:

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        int arr[] = {1, 0, 2, 0, 4, 0, 5};

        int index = 0;

        // Move non-zero elements forward

        for (int i = 0; i < arr.length; i++) {

            if (arr[i] != 0) {

                arr[index] = arr[i];

                index++;

            }

        }

        // Fill remaining positions with 0

        while (index < arr.length) {

            arr[index] = 0;

            index++;

        }

        for (int num : arr) {

            System.out.print(num + " ");

        }

    }

}
```

OUTPUT:

1 2 4 5 0 0 0

with two pointer

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter n:");

        int n = sc.nextInt();

        int arr[] = new int[n];
```

```

System.out.println("Enter elements:");
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}
int left = 0;
int right = n - 1;
while (left < right) {
    while (left < right && arr[left] != 0) {
        left++;
    }
    while (left < right && arr[right] == 0) {
        right--;
    }
    if (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
    }
}
System.out.println("After moving zeroes:");
for (int num : arr) {
    System.out.print(num + " ");
}
}

```

OUTPUT:

Enter n:

5

Enter elements:

1

0

5

0

6

After moving zeroes:

1 6 5 0 0

MAXIMUM SUM OF A SUBARRAY OF SIZE K USING SLIDING WINDOW

How it works:

Window 1 → [2 1 5] → sum = 8

Window 2 → [1 5 1] → remove 2, add 1 → sum = 7

Window 3 → [5 1 3] → remove 1, add 3 → sum = 9

Window 4 → [1 3 2] → remove 5, add 2 → sum = 6

```
class Main {  
    public static void main(String[] args) {  
        int arr[] = {-2, 1, -3, 4, -1, 2, 5};  
        int k = 3;  
        int max = 0;  
        for (int i = 0; i <= arr.length - k; i++) {  
            int sum = 0;  
            for (int j = i; j < i + k; j++) {  
                sum += arr[j];  
            }  
            if (sum > max) {  
                max = sum;  
            }  
        }  
        System.out.println("Maximum sum = " + max);  
    }  
}
```

MAXIMUM SUM OF A SUBARRAY OF SIZE K USING SLIDING WINDOW-OPTIMISED:

```
class Main {  
    public static void main(String[] args) {  
        int arr[] = {-2, 1, -3, 4, -1, 2, 5};  
        int k = 3;  
        int sum = 0;  
        for(int i=0;i<k;i++){  
            sum+=arr[i];  
        }  
        int maxx = sum;
```

```

        for (int i = k; i < arr.length; i++) {
            sum =sum-arr[i-k]+arr[i];
            maxx=Math.max(maxx, sum);
        }
        System.out.println("Maximum sum = " + maxx);
    }
}

```

PRINT THE SUM OF EVERY WINDOW:

```

class Main {
    public static void main(String[] args) {
        int arr[] = {-2, 1, -3, 4, -1, 2, 5};
        int k = 3;
        int sum = 0;
        a
        for(int i=0;i<k;i++){
            sum+=arr[i];
        }
        System.out.println(""+sum);
        int maxx = sum;
        for (int i = k; i < arr.length; i++) {
            sum =sum-arr[i-k]+arr[i];
            System.out.println(""+sum);
            //maxx=Math.max(maxx, sum);
        }
        //System.out.println("Maximum sum = " + maxx);
    }
}

```

MAXIMUM SUM OF A SUBARRAY USING SLIDING WINDOW (without size)

```

public class Main {
    public static void main(String[] args) {
        int arr[] = {-2, 1, -3, 4, -1, 2, 5, 4};
        int maxSum = arr[0];
        int currentSum = arr[0];
        for (int i = 1; i < arr.length; i++) {
            currentSum = Math.max(arr[i], currentSum + arr[i]);
        }
    }
}

```

```

        maxSum = Math.max(maxSum, currentSum);
    }

    System.out.println("Maximum subarray sum = " + maxSum);
}
}

```

OUTPUT:

Maximum subarray sum = 14

Brute force method:

```

public class Main {

    public static void main(String[] args) {

        int arr[] = {-2, 1, -3, 4, -1, 2, 5, 4};

        int maxsum = arr[0];

        for (int i = 1; i < arr.length; i++) {

            int sum=arr[i];

            maxsum=Math.max(maxsum,sum);

            for(int j=i+1;j<arr.length;j++){

                sum+=arr[j];

                maxsum=Math.max(maxsum,sum);

            }

        }

        System.out.println("Maximum subarray sum = " + maxsum);

    }

}

```

KADANE'S ALGORITHM:

```

public class Main {

    public static void main(String[] args) {

        int arr[] = {-2, 1, -3, 4, -1, 2, 5, 4};

        int maxsum = arr[0];

        int sum=0;

        for (int i = 1; i < arr.length; i++) {

            sum+=arr[i];

            maxsum=Math.max(maxsum,sum);

            if (sum<0){

                sum==0;

            }

        }

    }

}

```

```

    }

    System.out.println("Maximum subarray sum = " + maxsum);

}

}

```

OUTPUT:

Maximum subarray sum = 14

STRINGS

PRINT THE LENGTH OF THE LAST STRING

```

import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String a=sc.nextLine();
        String[] b=a.toLowerCase().trim().split(" ");
        System.out.println(b[b.length-1].length());
    }
}

```

OUTPUT:

HELLO WORLD

5

INPUT: C5D2A3

OUTPUT: CCCCCDDAAA

```

import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        for (int i = 0; i < s.length(); ) {
            char ch = s.charAt(i);
            i++;
            String num = "";
            while (i < s.length() && Character.isDigit(s.charAt(i))) {
                num += s.charAt(i);
                i++;
            }
            int count = Integer.parseInt(num);
            for (int j = 0; j < count; j++) {
                System.out.print(ch);
            }
        }
    }
}

```